```
const mineflayer = require('mineflayer');
const express = require('express');
const { SocksClient } = require('socks');

const app = express();
const port = process.env.PORT || 10000;

app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// ÉP MÚI GIỜ VIỆT NAM TOÀN HỆ THỐNG
process.env.TZ = 'Asia/Ho_Chi_Minh';

let BOT_USERNAME = process.env.BOT_USERNAME || 'Kiru Đẹp Trai';
let BOT_PASSWORD = process.env.BOT_PASSWORD || 'YourPasswordHere';
let BOT_HOST = process.env.BOT_HOST || 'mc.example.com';
let BOT_PORT = parseInt(process.env.BOT_PORT) || 25565;
let BOT_PROXY = process.env.BOT_PROXY || ''; // Định dạng: host:port hoặc
user:pass@host:port (SOCKS5)

// --- BẬT/TẮT CÁC TÍNH NĂNG TOGGLE ---
const toggles = {
  afkmode: false,
  thien: false,
  quylay: false,
  dinhthan: false,
  quanghao: false,
  ngoi: false,     // Nút ngồi (Sneak)
  attackLeft: false,
  attackRight: false,
  skill1: false,
  skill2: false,
  skill3: false
};

// Cấu hình tốc độ click chuột (tối thiểu 0.1s = 100ms)
let attackLeftIntervalMs = 500;
let attackRightIntervalMs = 500;

// CẤU HÌNH Ô CÔNG CỤ (HOTBAR 1 -> 9 tương ứng Index 0 -> 8)
let hotbarEnabled = [false, false, false, false, false, false, false, false, false];
let hotbarHoldDurationSec = 5;
let currentHotbarCycleIndex = 0;
```

```
let bot = null;
let reconnectTimeout = null;
let isReconnecting = false;
let isManualStopped = false;
let isFirstSpawn = true;

let currentReconnectDelay = 10000;
let consecutiveFailures = 0;

// TIMERS & INTERVALS
let actionTimeout = null;
let antiAfkTimeout = null;
let ramGcInterval = null;
let posCheckInterval = null;
let watchdogInterval = null;
let pingInterval = null;
let loginTimer1 = null;
let loginTimer2 = null;
let respawnTimer = null;
let commandResponseTimer = null;

let quylayInterval = null;
let ngoiInterval = null;
let attackLeftInterval = null;
let attackRightInterval = null;
let skillLoopInterval = null;
let hotbarLoopInterval = null;

let isAwaitingResponse = false;
let isAutoActionRunning = false;
let lastActionTime = Date.now();
let currentCoords = 'Đang xác định...';
let currentPing = 0;

const startTime = Date.now();
const serverChatLogs = [];
const errorLogs = [];
const pingLogs = [];
const botMentionLogs = [];

let lastTimeAge = 0;
let lastTimeAgeUpdate = Date.now();
```

```
function getVNTime() {
  return new Date().toLocaleTimeString('vi-VN', { timeZone: 'Asia/Ho_Chi_Minh', hour12: false });
}

function addChatLog(msg) {
  serverChatLogs.unshift(`[${getVNTime()}] ${msg}`);
  if (serverChatLogs.length > 30) serverChatLogs.pop();
}

function addErrorLog(type, details) {
  errorLogs.unshift({ time: getVNTime(), type: type, details: details });
  if (errorLogs.length > 30) errorLogs.pop();
}

function addPingLog(pingVal) {
  pingLogs.unshift({ time: getVNTime(), ping: pingVal });
  if (pingLogs.length > 15) pingLogs.pop();
}

function addBotMentionLog(msg) {
  botMentionLogs.unshift({ time: getVNTime(), text: msg });
  if (botMentionLogs.length > 20) botMentionLogs.pop();
}

function triggerChatWindow(durationMs = 8000) {
  isAwaitingResponse = true;
  if (commandResponseTimer) clearTimeout(commandResponseTimer);
  commandResponseTimer = setTimeout(() => { isAwaitingResponse = false; }, durationMs);
}

function getHeldItemName() {
  if (!bot || !bot.heldItem) return 'Tay không';
  const item = bot.heldItem;
  return item.customName || item.displayName || item.name || 'Vật phẩm không tên';
}

function safeChat(msg) {
  if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
    try {
      bot.chat(msg);
      addChatLog(`[TỰ ĐỘNG]: ${msg}`);
    } catch (e) {
```

```
      addErrorLog('Lỗi Chat', e.message);
    }
  }
}

app.get('/api/ping', (req, res) => res.send('PONG_OK'));

// ENDPOINT CẬP NHẬT CẤU HÌNH (USERNAME, PASSWORD, HOST, PROXY)
app.post('/api/update-config', (req, res) => {
  try {
    const { username, password, host, proxy } = req.body;

    if (username !== undefined && username.trim() !== '') BOT_USERNAME = username.trim();
    if (password !== undefined && password.trim() !== '') BOT_PASSWORD = password.trim();
    if (proxy !== undefined) BOT_PROXY = proxy.trim();

    if (host !== undefined && host.trim() !== '') {
      let cleanHost = host.trim();
      if (cleanHost.includes(':')) {
        const parts = cleanHost.split(':');
        BOT_HOST = parts[0];
        BOT_PORT = parseInt(parts[1]) || 25565;
      } else {
        BOT_HOST = cleanHost;
        BOT_PORT = 25565;
      }
    }

    addErrorLog('CẤU HÌNH', `Đã lưu cấu hình Bot [${BOT_USERNAME}]. Đang tái kết nối...`);
    consecutiveFailures = 0;
    isManualStopped = false;
    createBot();
  } catch (err) {
    addErrorLog('Lỗi Lưu Cấu Hình', err.message);
  }
  res.redirect('/');
});

app.post('/api/command', (req, res) => {
  if (isManualStopped) return res.send('Bot đang ở trạng thái TẮT thủ công!');
  const cmd = req.body.command;
  if (!bot || !bot._client) return res.send('Bot đang ngoại tuyến!');
  if (cmd) {
```

```
    bot.chat(cmd);
    addChatLog(`[WEB-ADMIN]: ${cmd}`);
    triggerChatWindow(8000);
  }
  res.redirect('/');
});

app.get('/api/toggle/:feature', (req, res) => {
  const feat = req.params.feature;
  if (toggles.hasOwnProperty(feat)) {
    toggles[feat] = !toggles[feat];
    addChatLog(`[CÀI ĐẶT] ${feat.toUpperCase()} ➔ ${toggles[feat] ? 'BẬT' : 'TẮT'}`);

    if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
      if (feat === 'afkmode') safeChat(toggles[feat] ? '/afkmode vao' : '/afkmode ra');
      if (feat === 'thien') safeChat('/thien');
      if (feat === 'quylay') safeChat('/quylay');
      if (feat === 'dinhthan') safeChat('/dinhthan');
      if (feat === 'quanghao') safeChat('/quanghao');
      if (feat === 'skill1') safeChat('/kinang_1');
      if (feat === 'skill2') safeChat('/kinang_2');
      if (feat === 'skill3') safeChat('/kinang_3');

      restartLoops();
    }
  }
  res.redirect('/');
});

app.post('/api/update-hotbar', (req, res) => {
  const duration = parseInt(req.body.duration);
  if (!isNaN(duration) && duration >= 1) hotbarHoldDurationSec = duration;

  for (let i = 0; i < 9; i++) {
    hotbarEnabled[i] = req.body[`slot_${i}`] === 'on';
  }

  addChatLog(`[HOTBAR] Cập nhật danh sách ô cầm vật phẩm (${hotbarHoldDurationSec}s)`);
  restartLoops();
  res.redirect('/');
});

app.post('/api/update-click-speed', (req, res) => {
```

```
  const leftSpeed = parseFloat(req.body.leftSpeed);
  const rightSpeed = parseFloat(req.body.rightSpeed);

  if (!isNaN(leftSpeed) && leftSpeed >= 0.1) attackLeftIntervalMs = Math.round(leftSpeed *
1000);
  if (!isNaN(rightSpeed) && rightSpeed >= 0.1) attackRightIntervalMs = Math.round(rightSpeed *
1000);

  restartLoops();
  res.redirect('/');
});

app.get('/api/inventory', (req, res) => {
  if (bot && bot._client && !isManualStopped) {
    bot.chat('[inv]');
    addChatLog('[WEB-ADMIN]: [inv]');
    triggerChatWindow(4000);
  }
  res.redirect('/');
});

app.get('/api/tusat', (req, res) => {
  if (bot && bot._client && !isManualStopped) {
    safeChat('/tusat');
    triggerChatWindow(4000);
  }
  res.redirect('/');
});

app.get('/api/toggle-bot', (req, res) => {
  isManualStopped = !isManualStopped;
  if (isManualStopped) {
    addErrorLog('THỦ CÔNG', 'Đã TẮT Bot từ Dashboard. Ngắt kết nối.');
    cleanupBot();
  } else {
    addErrorLog('THỦ CÔNG', 'Đã BẬT lại Bot từ Dashboard. Đang kết nối lại...');
    consecutiveFailures = 0;
    createBot();
  }
  res.redirect('/');
});

app.get('/api/clear-error-log', (req, res) => { errorLogs.length = 0; res.redirect('/'); });
```

```
app.get('/api/clear-mention-log', (req, res) => { botMentionLogs.length = 0; res.redirect('/'); });
app.get('/api/hard-restart', (req, res) => { process.exit(1); });

app.get('/', (req, res) => {
  const uptimeMinutes = Math.floor((Date.now() - startTime) / 60000);
  const memoryUsage = (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
  const currentWeapon = getHeldItemName();

  let statusBadge = '<span class="badge-off">OFFLINE</span>';
  if (isManualStopped) {
    statusBadge = '<span class="badge-pause">ĐÃ TẮT THỦ CÔNG</span>';
  } else if (bot && bot._client && bot._client.state === 'play') {
    statusBadge = '<span class="badge-on">ONLINE</span>';
  } else {
    statusBadge = `<span class="badge-off">RECONNECTING
(${Math.round(currentReconnectDelay / 1000)}s)</span>`;
  }

  const renderToggleBtn = (key, label) => {
    const isON = toggles[key];
    return `<a href="/api/toggle/${key}" style="text-decoration: none;"><button type="button"
class="${isON ? 'btn-start' : 'btn-stop'}" style="width: 100%;">${label}: ${isON ? 'BẬT' :
'TẮT'}</button></a>`;
  };

  res.send(`
    <!DOCTYPE html>
    <html lang="vi">
    <head>
      <meta charset="utf-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0,
user-scalable=no">
      <title>Kiru Đẹp Trai - Control Panel</title>
      <link
href="https://fonts.googleapis.com/css2?family=Orbitron:wght@600;800&family=Plus+Jakarta+
Sans:wght@400;600;700&display=swap" rel="stylesheet">
      <style>
        :root {
          --card-bg: rgba(10, 14, 26, 0.82);
          --accent-cyan: #38bdf8;
          --accent-pink: #f43f5e;
          --accent-purple: #c084fc;
          --accent-green: #4ade80;
```

```
      --accent-yellow: #fbbf24;
      --border: rgba(56, 189, 248, 0.35);
      --glow: 0 0 15px rgba(56, 189, 248, 0.25);
    }
    * { box-sizing: border-box; }
    body {
      font-family: 'Plus Jakarta Sans', sans-serif;
      margin: 0;
      padding: 16px;
      color: #f8fafc;
      min-height: 100vh;
      background: radial-gradient(circle at 50% 10%, #0f172a 0%, #020617 100%);
      background-attachment: fixed;
    }
    .toggle-ui-btn {
      position: fixed; bottom: 20px; right: 20px; z-index: 9999;
      padding: 12px 20px; background: linear-gradient(135deg, #0ea5e9, #38bdf8);
      color: #fff; font-weight: 800; border: 2px solid #38bdf8; border-radius: 30px;
      box-shadow: 0 0 20px rgba(56, 189, 248, 0.6); cursor: pointer; font-size: 0.95rem;
      display: flex; align-items: center; gap: 8px; transition: transform 0.2s;
    }
    .toggle-ui-btn:hover { transform: scale(1.05); }
    .header { display: flex; align-items: center; justify-content: space-between; padding-bottom:
12px; border-bottom: 2px solid var(--border); margin-bottom: 16px; gap: 12px; flex-wrap: wrap; }
    h1 {
      font-family: 'Orbitron', sans-serif; font-size: 1.8rem; margin: 0;
      background: linear-gradient(90deg, #38bdf8, #f43f5e, #c084fc);
      -webkit-background-clip: text; -webkit-text-fill-color: transparent;
      text-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
    }
    .container { display: grid; grid-template-columns: 2fr 1fr; gap: 16px; }
    .card { background: var(--card-bg); backdrop-filter: blur(14px); padding: 16px;
border-radius: 16px; border: 1px solid var(--border); box-shadow: var(--glow); margin-bottom:
16px; }
    h3 { margin-top: 0; color: var(--accent-cyan); font-size: 1.05rem; display: flex; align-items:
center; gap: 8px; }
    .badge-on { background: rgba(74, 222, 128, 0.25); color: #4ade80; border: 1px solid
#22c55e; padding: 6px 14px; border-radius: 20px; font-weight: bold; font-size: 0.85rem; }
    .badge-off { background: rgba(244, 63, 94, 0.25); color: #f43f5e; border: 1px solid #f43f5e;
padding: 6px 14px; border-radius: 20px; font-weight: bold; font-size: 0.85rem; }
    .badge-pause { background: rgba(251, 191, 36, 0.25); color: #fbbf24; border: 1px solid
#f59e0b; padding: 6px 14px; border-radius: 20px; font-weight: bold; font-size: 0.85rem; }
    .status-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
```

```
gap: 8px; margin: 12px 0; }
        .status-item { background: rgba(0, 0, 0, 0.5); border: 1px solid rgba(255, 255, 255, 0.12);
padding: 8px 10px; border-radius: 10px; display: flex; flex-direction: column; gap: 2px; }
        .status-item .label { font-size: 0.68rem; color: #94a3b8; text-transform: uppercase;
font-weight: 700; }
        .status-item .value { font-size: 0.85rem; font-weight: 700; color: #f8fafc; word-break:
break-all; }
        .chat-box { background: rgba(0, 0, 0, 0.7); padding: 10px; border-radius: 10px; font-family:
monospace; height: 260px; overflow-y: auto; color: #38bdf8; border: 1px solid rgba(56, 189,
248, 0.3); font-size: 0.8rem; }
        .error-box { background: rgba(20, 5, 5, 0.75); padding: 10px; border-radius: 10px;
font-family: monospace; height: 180px; overflow-y: auto; color: #f87171; border: 1px solid
rgba(244, 63, 94, 0.4); font-size: 0.8rem; }
        .kiru-box { background: rgba(15, 23, 15, 0.75); padding: 10px; border-radius: 10px;
font-family: monospace; height: 150px; overflow-y: auto; color: #facc15; border: 1px solid
rgba(250, 204, 21, 0.4); font-size: 0.8rem; }
        .input-group { display: flex; gap: 8px; margin-top: 8px; }
        .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
        input[type="text"], input[type="password"], input[type="number"] {
          width: 100%; padding: 8px 12px; border-radius: 8px; border: 1px solid rgba(255, 255,
255, 0.2);
          background: rgba(0, 0, 0, 0.6); color: white; outline: none; font-size: 0.85rem;
        }
        input:focus { border-color: var(--accent-pink); box-shadow: 0 0 10px rgba(244, 63, 94, 0.5);
}
        button { padding: 9px 15px; background: linear-gradient(135deg, #e11d48, #be123c); color:
white; border: 1px solid rgba(255,255,255,0.2); border-radius: 8px; cursor: pointer; font-weight:
bold; transition: all 0.2s ease; white-space: nowrap; font-size: 0.85rem; }
        button:hover { transform: translateY(-2px); box-shadow: 0 4px 15px rgba(225, 29, 72, 0.5);
}
        .btn-stop { background: linear-gradient(135deg, #dc2626, #991b1b) !important; }
        .btn-start { background: linear-gradient(135deg, #16a34a, #15803d) !important; }
        .btn-warning { background: linear-gradient(135deg, #d97706, #b45309) !important; }
        .btn-purple { background: linear-gradient(135deg, #9333ea, #6b21a8) !important; }
        .btn-save { background: linear-gradient(135deg, #0284c7, #0369a1) !important; width:
100%; margin-top: 8px; }
        label { font-size: 0.78rem; color: #cbd5e1; display: block; margin-bottom: 4px; font-weight:
600; }
        .btn-group-responsive { display: grid; grid-template-columns: repeat(auto-fit,
minmax(120px, 1fr)); gap: 8px; margin-top: 10px; }
        .hotbar-grid { display: grid; grid-template-columns: repeat(9, 1fr); gap: 6px; margin-top:
10px; }
        .hotbar-item { background: rgba(0,0,0,0.6); border: 1px solid rgba(255,255,255,0.15);
```

```
padding: 8px 4px; border-radius: 8px; text-align: center; }
      .hotbar-item input[type="checkbox"] { width: 16px; height: 16px; cursor: pointer; }
      @media (max-width: 1024px) { .container { grid-template-columns: 1fr; } .hotbar-grid {
grid-template-columns: repeat(5, 1fr); } }
    </style>
    <script>
      function toggleDashboardView() {
        const container = document.getElementById('main-dashboard');
        const toggleBtn = document.getElementById('toggle-ui-text');
        if (container.style.display === 'none') {
          container.style.display = 'grid';
          toggleBtn.textContent = 'Thu Gọn Bảng Control';
          localStorage.setItem('ui_collapsed', 'false');
        } else {
          container.style.display = 'none';
          toggleBtn.textContent = 'Hiện Bảng Control';
          localStorage.setItem('ui_collapsed', 'true');
        }
      }
      window.addEventListener('DOMContentLoaded', () => {
        if (localStorage.getItem('ui_collapsed') === 'true') {
          document.getElementById('main-dashboard').style.display = 'none';
          document.getElementById('toggle-ui-text').textContent = 'Hiện Bảng Control';
        }
        setInterval(() => {
          const activeEl = document.activeElement;
          if (!activeEl || (activeEl.tagName !== 'INPUT' && activeEl.tagName !== 'TEXTAREA')) {
            location.reload();
          }
        }, 12000);
      });
    </script>
  </head>
  <body>
    <button type="button" class="toggle-ui-btn" onclick="toggleDashboardView()">
      ⚙️ <span id="toggle-ui-text">Thu Gọn Bảng Control</span>
    </button>
    <div class="header">
      <h1>KIRU ĐẸP TRAI</h1>
      <div>${statusBadge}</div>
    </div>
    <div class="container" id="main-dashboard">
      <div>
```

```
        <div class="card">
          <h3>Trạng Thái Bot: <span style="color:
var(--accent-pink);">${BOT_USERNAME}</span></h3>
          <div class="status-grid">
            <div class="status-item"><span class="label">Server</span><span
class="value"><code>${BOT_HOST}:${BOT_PORT}</code></span></div>
            <div class="status-item"><span class="label">Proxy IP</span><span class="value"
style="color:var(--accent-yellow);"><code>${BOT_PROXY ? BOT_PROXY : 'Trực tiếp (Render
IP)'}</code></span></div>
            <div class="status-item"><span class="label">Ping</span><span class="value"
style="color: var(--accent-cyan);">${currentPing} ms</span></div>
            <div class="status-item"><span class="label">Tọa Độ</span><span
class="value"><code>${currentCoords}</code></span></div>
            <div class="status-item"><span class="label">Vật Phẩm Trên Tay</span><span
class="value" style="color:
var(--accent-yellow);"><code>${currentWeapon}</code></span></div>
            <div class="status-item"><span class="label">Hoạt Động</span><span
class="value">${uptimeMinutes} phút</span></div>
            <div class="status-item"><span class="label">RAM</span><span
class="value">${memoryUsage} MB</span></div>
          </div>
          <form class="input-group" action="/api/command" method="POST">
            <input type="text" name="command" placeholder="Gửi lệnh hoặc chat vào server..."
autocomplete="off" required>
            <button type="submit">Gửi Chat</button>
          </form>
          <div class="btn-group-responsive">
            ${isManualStopped
              ? `<a href="/api/toggle-bot" style="text-decoration: none;"><button type="button"
class="btn-start" style="width: 100%;">BẬT BOT</button></a>`
              : `<a href="/api/toggle-bot" style="text-decoration: none;"><button type="button"
class="btn-stop" style="width: 100%;">TẮT BOT</button></a>`
            }
            <a href="/api/inventory" style="text-decoration: none;"><button type="button"
class="btn-purple" style="width: 100%;">Túi Đồ [inv]</button></a>
            <a href="/api/tusat" style="text-decoration: none;"><button type="button"
class="btn-stop" style="width: 100%;">Tự Sát (/tusat)</button></a>
            <a href="/api/clear-error-log" style="text-decoration: none;"><button type="button"
class="btn-warning" style="width: 100%;">Xóa Lỗi</button></a>
            <a href="/api/clear-mention-log" style="text-decoration: none;"><button type="button"
class="btn-warning" style="width: 100%;">Mention (${botMentionLogs.length})</button></a>
            <a href="/api/hard-restart" style="text-decoration: none;" onclick="return confirm('Reset
App?');"><button type="button" class="btn-stop" style="width: 100%;">Reset App</button></a>
```

```
          </div>
        </div>

        <!-- CẤU HÌNH Ô CÔNG CỤ -->
        <div class="card">
          <h3>Cấu Hình Ô Công Cụ Bot (Ô 1 ➔ Ô 9)</h3>
          <form action="/api/update-hotbar" method="POST">
            <div><label>Thời gian cầm mỗi ô (giây):</label><input type="number" min="1"
name="duration" value="${hotbarHoldDurationSec}" required></div>
            <div style="margin-top: 8px;"><label>Chọn ô cần luân chuyển:</label></div>
            <div class="hotbar-grid">
              ${
  [1,2,3,4,5,6,7,8,9].map(function(num, i) {
    return '<div class="hotbar-item"><label style="margin-bottom:2px;">Ô ' + num +
'</label><input type="checkbox" name="slot_' + i + '" ' + (hotbarEnabled[i] ? 'checked' : '') +
'></div>';
  }).join('')
}
            </div>
            <button type="submit" class="btn-save">Lưu Thiết Lập Ô Cầm</button>
          </form>
        </div>

        <!-- BẬT/TẮT TÍNH NĂNG AUTO -->
        <div class="card">
          <h3>Bật / Tắt Lệnh Tự Động & Hoạt Động</h3>
          <div class="btn-group-responsive">
            ${renderToggleBtn('afkmode', 'AFK Mode')}
            ${renderToggleBtn('thien', 'Thiền')}
            ${renderToggleBtn('quylay', 'Quỳ Lạy')}
            ${renderToggleBtn('dinhthan', 'Định Thần')}
            ${renderToggleBtn('quanghao', 'Quang Hào')}
            ${renderToggleBtn('ngoi', 'Ngồi / Rón Rén')}
          </div>
        </div>

        <!-- ĐÁNH TRÁI / PHẢI -->
        <div class="card">
          <h3>Đánh Liên Tục (Click Mouse)</h3>
          <div class="btn-group-responsive" style="margin-bottom: 10px;">
            ${renderToggleBtn('attackLeft', 'Đánh Trái')}
            ${renderToggleBtn('attackRight', 'Đánh Phải')}
          </div>
```

```
          <form action="/api/update-click-speed" method="POST" class="form-grid">
            <div><label>Tốc độ Trái (giây):</label><input type="number" step="0.1" min="0.1"
name="leftSpeed" value="${(attackLeftIntervalMs / 1000).toFixed(1)}"></div>
            <div><label>Tốc độ Phải (giây):</label><input type="number" step="0.1" min="0.1"
name="rightSpeed" value="${(attackRightIntervalMs / 1000).toFixed(1)}"></div>
            <div style="grid-column: 1 / -1;"><button type="submit" class="btn-save">Lưu Tốc Độ
Click</button></div>
          </form>
        </div>

        <!-- SKILL -->
        <div class="card">
          <h3>Khu Vực Skill</h3>
          <div class="btn-group-responsive">
            ${renderToggleBtn('skill1', 'Kỹ Năng 1')}
            ${renderToggleBtn('skill2', 'Kỹ Năng 2')}
            ${renderToggleBtn('skill3', 'Kỹ Năng 3')}
          </div>
        </div>

        <!-- CẤU HÌNH TÀI KHOẢN & PROXY -->
        <form action="/api/update-config" method="POST">
          <div class="card">
            <h3>Cấu Hình Đăng Nhập, Server & Proxy</h3>
            <div class="form-grid">
              <div><label>Tên Nhân Vật:</label><input type="text" name="username"
value="${BOT_USERNAME}" required autocomplete="off"></div>
              <div><label>Mật Khẩu:</label><input type="password" name="password"
value="${BOT_PASSWORD}" required autocomplete="off"></div>
            </div>
            <div style="margin-top: 10px;">
              <label>IP Server (Host:Port):</label>
              <input type="text" name="host" value="${BOT_HOST}${BOT_PORT && BOT_PORT
!== 25565 ? ':' + BOT_PORT : ''}" required autocomplete="off">
            </div>
            <div style="margin-top: 10px;">
              <label>SOCKS5 Proxy (Bỏ trống nếu dùng trực tiếp Render IP):</label>
              <input type="text" name="proxy" value="${BOT_PROXY}" placeholder="ip:port hoặc
username:password@ip:port" autocomplete="off">
            </div>
            <button type="submit" class="btn-save">Lưu Cấu Hình & Kết Nối Lại</button>
          </div>
        </form>
```

```
        <div class="card">
          <h3>Nhật Ký Nhắc Tên [${BOT_USERNAME}]</h3>
          <div class="kiru-box">
            ${botMentionLogs.length > 0 ? botMentionLogs.map(function(k) { return "<div>[" +
k.time + "] " + k.text + "</div>"; }).join('') : '<i>Chưa có tin nhắn nào...</i>'}
          </div>
        </div>
      </div>

      <div>
        <div class="card">
          <h3>Lịch Sử Ping</h3>
          <p style="word-break: break-all; margin:0;"><code>${pingLogs.length > 0 ?
pingLogs.map(function(p) { return "[" + p.time + ":" + p.ping + "ms]"; }).join(' ➔ ') : 'Đang thu
thập...'}</code></p>
        </div>
        <div class="card">
          <h3>Chat Server (Realtime)</h3>
          <div class="chat-box">
            ${serverChatLogs.length > 0 ? serverChatLogs.map(function(l) { return "<div>" + l +
"</div>"; }).join('') : '<i>Chưa có nhật ký...</i>'}
          </div>
        </div>
        <div class="card">
          <h3>Nhật Ký Lỗi Phát Sinh</h3>
          <div class="error-box">
            ${errorLogs.length > 0 ? errorLogs.map(function(e) { return "<div>[" + e.time + "] <b>["
+ e.type + "]</b>: " + e.details + "</div>"; }).join('') : '<div
style="color:var(--accent-green);">Không có lỗi!</div>'}
          </div>
        </div>
      </div>
    </div>
  </body>
  </html>
 `);
});

app.listen(port, () => console.log(`[HTTP SERVER] Running on port ${port}`));

function stopFeatureLoops() {
 if (quylayInterval) { clearInterval(quylayInterval); quylayInterval = null; }
```

```
  if (ngoiInterval) { clearInterval(ngoiInterval); ngoiInterval = null; }
  if (attackLeftInterval) { clearInterval(attackLeftInterval); attackLeftInterval = null; }
  if (attackRightInterval) { clearInterval(attackRightInterval); attackRightInterval = null; }
  if (skillLoopInterval) { clearInterval(skillLoopInterval); skillLoopInterval = null; }
  if (hotbarLoopInterval) { clearInterval(hotbarLoopInterval); hotbarLoopInterval = null; }
}

function restartLoops() {
  stopFeatureLoops();
  if (!bot || !bot.entity || !bot._client || bot._client.state !== 'play' || isManualStopped) return;

  if (toggles.quylay) {
    quylayInterval = setInterval(() => { if (toggles.quylay) safeChat('/quylay'); }, 47000);
  }
  if (toggles.ngoi) {
    ngoiInterval = setInterval(() => {
      if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
        try {
          bot.setControlState('sneak', true);
          setTimeout(() => { if (bot && bot.entity) bot.setControlState('sneak', false); }, 1200);
        } catch (e) {}
      }
    }, 5000);
  }
  if (toggles.attackLeft) {
    attackLeftInterval = setInterval(() => {
      if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
        try { bot.swingArm('right'); } catch (e) {}
      }
    }, attackLeftIntervalMs);
  }
  if (toggles.attackRight) {
    attackRightInterval = setInterval(() => {
      if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
        try { bot.activateItem(); } catch (e) {}
      }
    }, attackRightIntervalMs);
  }
  if (toggles.skill1 || toggles.skill2 || toggles.skill3) {
    skillLoopInterval = setInterval(() => {
      if (bot && bot.entity && bot._client && !isManualStopped) {
        if (toggles.skill1) safeChat('/kinang_1');
        if (toggles.skill2) safeChat('/kinang_2');
```

```
        if (toggles.skill3) safeChat('/kinang_3');
      }
    }, 10000);
  }

  const activeSlots = hotbarEnabled.map((enabled, idx) => enabled ? idx : -1).filter(idx => idx !==
-1);
  if (activeSlots.length === 1) {
    try { bot.setQuickBarSlot(activeSlots[0]); } catch (e) {}
  } else if (activeSlots.length > 1) {
    hotbarLoopInterval = setInterval(() => {
      if (bot && bot.entity && bot._client && bot._client.state === 'play' && !isManualStopped) {
        currentHotbarCycleIndex = (currentHotbarCycleIndex + 1) % activeSlots.length;
        try { bot.setQuickBarSlot(activeSlots[currentHotbarCycleIndex]); } catch (e) {}
      }
    }, hotbarHoldDurationSec * 1000);
  }
}

function cleanupBot() {
  isAutoActionRunning = false;
  currentCoords = 'Đang xác định...';
  currentPing = 0;

  stopFeatureLoops();

  if (actionTimeout) { clearTimeout(actionTimeout); actionTimeout = null; }
  if (antiAfkTimeout) { clearTimeout(antiAfkTimeout); antiAfkTimeout = null; }

  [ramGcInterval, posCheckInterval, watchdogInterval, pingInterval].forEach(i => i &&
clearInterval(i));
  [reconnectTimeout, loginTimer1, loginTimer2, respawnTimer,
commandResponseTimer].forEach(t => t && clearTimeout(t));

  if (bot) {
    try {
      bot.removeAllListeners();
      if (bot._client) bot._client.removeAllListeners();
      bot.clearControlStates();
      bot.end();
    } catch (e) {}
    bot = null;
  }
```

```
  if (global.gc) {
    try { global.gc(); } catch (e) {}
  }
}

function scheduleNextAction() {
  if (actionTimeout) { clearTimeout(actionTimeout); actionTimeout = null; }
  if (isManualStopped || !bot || !bot.entity || !bot._client) {
    isAutoActionRunning = false;
    return;
  }
  isAutoActionRunning = true;
  lastActionTime = Date.now();
  const currentBot = bot;

  try {
    const actionType = Math.floor(Math.random() * 3);
    if (actionType === 0) currentBot.swingArm('right');
    else if (actionType === 1) {
      currentBot.setControlState('sneak', true);
      setTimeout(() => { if (bot === currentBot && bot.entity) bot.setControlState('sneak', false); },
200);
    } else {
      currentBot.activateItem();
    }
  } catch (err) {}

  actionTimeout = setTimeout(scheduleNextAction, Math.floor(5000 + Math.random() * 5000));
}

function scheduleRandomRotation() {
  if (antiAfkTimeout) clearTimeout(antiAfkTimeout);
  if (isManualStopped) return;

  antiAfkTimeout = setTimeout(() => {
    if (bot && bot.entity && bot.health > 0 && bot._client && bot._client.state === 'play') {
      try {
        bot.look(bot.entity.yaw + (Math.random() - 0.5) * 0.3, bot.entity.pitch + (Math.random() -
0.5) * 0.1, true);
      } catch (e) {}
    }
    scheduleRandomRotation();
```

```
  }, Math.floor(15000 + Math.random() * 20000));
}

function executeActiveFeaturesOnSpawn() {
  if (!bot || !bot.entity || !bot._client || isManualStopped) return;
  let delay = 1000;

  if (toggles.afkmode) { setTimeout(() => safeChat('/afkmode vao'), delay); delay += 1200; }
  if (toggles.thien) { setTimeout(() => safeChat('/thien'), delay); delay += 1200; }
  if (toggles.quylay) { setTimeout(() => safeChat('/quylay'), delay); delay += 1200; }
  if (toggles.dinhthan) { setTimeout(() => safeChat('/dinhthan'), delay); delay += 1200; }
  if (toggles.quanghao) { setTimeout(() => safeChat('/quanghao'), delay); delay += 1200; }

  setTimeout(() => { restartLoops(); }, delay + 500);
}

function createBot() {
  if (isManualStopped) return;
  cleanupBot();
  isFirstSpawn = true;
  lastTimeAge = 0;
  lastTimeAgeUpdate = Date.now();

  const currentOptions = {
    host: BOT_HOST,
    port: BOT_PORT,
    username: BOT_USERNAME,
    hideErrors: false,
    checkTimeoutInterval: 60 * 1000,
    keepAlive: true,
    physicsEnabled: true,
    viewDistance: 'tiny'
  };

  // NẾU CÓ CẤU HÌNH PROXY SOCKS5 -> SỬ DỤNG PROXY ĐỂ TRÁNH LỖI TRÙNG IP
TRÊN RENDER
  if (BOT_PROXY && BOT_PROXY.trim() !== '') {
    currentOptions.connect = (client) => {
      let proxyHost = BOT_PROXY;
      let proxyPort = 1080;
      let proxyUser = undefined;
      let proxyPass = undefined;
```

```
    try {
      if (BOT_PROXY.includes('@')) {
        const parts = BOT_PROXY.split('@');
        const auth = parts[0].split(':');
        proxyUser = auth[0];
        proxyPass = auth[1];
        const serverParts = parts[1].split(':');
        proxyHost = serverParts[0];
        proxyPort = parseInt(serverParts[1]) || 1080;
      } else if (BOT_PROXY.includes(':')) {
        const parts = BOT_PROXY.split(':');
        proxyHost = parts[0];
        proxyPort = parseInt(parts[1]) || 1080;
      }

      SocksClient.createConnection({
        proxy: {
          host: proxyHost,
          port: proxyPort,
          type: 5,
          userId: proxyUser,
          password: proxyPass
        },
        command: 'connect',
        destination: {
          host: BOT_HOST,
          port: BOT_PORT
        }
      }, (err, info) => {
        if (err) {
          addErrorLog('Proxy Error', err.message);
          client.emit('error', err);
          return;
        }
        client.setSocket(info.socket);
        client.emit('connect');
      });
    } catch (proxyErr) {
      addErrorLog('Proxy Parse Error', proxyErr.message);
    }
  };
}
```

```
  console.log(`
[HỆ THỐNG] Kết nối đến ${BOT_HOST}:${BOT_PORT} (Proxy: ${BOT_PROXY || 'Không'}) với
tên [${BOT_USERNAME}]...`);

  try {
    bot = mineflayer.createBot(currentOptions);
    bot.setMaxListeners(0);

    if (bot._client) {
      bot._client.setMaxListeners(0);
      bot._client.on('error', (err) => {
        addErrorLog('Socket Error', err.message || err.code || 'Lỗi TCP Socket');
      });
    }
  } catch (err) {
    addErrorLog('Init Failed', err.message);
    handleReconnect();
    return;
  }

  bot.on('spawn', () => {
    console.log('[LOG] Bot đã vào game!');
    addChatLog('Kết nối ổn định thành công!');
    triggerChatWindow(12000);
    consecutiveFailures = 0;
    currentReconnectDelay = 10000;

    if (isFirstSpawn) {
      isFirstSpawn = false;

      loginTimer1 = setTimeout(() => {
        if (bot && bot._client && !isManualStopped) {
          bot.chat(`/l ${BOT_PASSWORD}`);
          triggerChatWindow(4000);
        }
      }, 3500);

      loginTimer2 = setTimeout(() => {
        if (bot && bot.entity && bot._client && !isManualStopped) {
          executeActiveFeaturesOnSpawn();
          scheduleNextAction();
          scheduleRandomRotation();
        }
```

```
    }, 7000);

    ramGcInterval = setInterval(() => {
      if (bot && bot.entities) {
        const myPos = bot.entity ? bot.entity.position : null;
        Object.keys(bot.entities).forEach(id => {
          const ent = bot.entities[id];
          if (!ent || !myPos || (ent.position && ent.position.distanceTo(myPos) > 12) || ent.id !==
bot.entity?.id) {
            delete bot.entities[id];
          }
        });
      }
      if (global.gc) { try { global.gc(); } catch (e) {} }
    }, 15000);

    pingInterval = setInterval(() => {
      if (bot && bot.player) {
        currentPing = bot.player.ping || 0;
        addPingLog(currentPing);
      }
    }, 10000);

    posCheckInterval = setInterval(() => {
      if (bot && bot.entity && bot.entity.position) {
        const pos = bot.entity.position;
        currentCoords = `X: ${pos.x.toFixed(1)}, Y: ${pos.y.toFixed(1)}, Z: ${pos.z.toFixed(1)}`;
      }
    }, 5000);

    watchdogInterval = setInterval(() => {
      if (!bot || isManualStopped) return;
      if (!isAutoActionRunning || Date.now() - lastActionTime > 20000) scheduleNextAction();

      if (bot.time) {
        if (bot.time.age === lastTimeAge) {
          if (Date.now() - lastTimeAgeUpdate > 45000) {
            addErrorLog('Watchdog', 'Kẹt Packet thế giới > 45s. Tiến hành Reconnect...');
            handleReconnect();
          }
        } else {
          lastTimeAge = bot.time.age;
          lastTimeAgeUpdate = Date.now();
```

```
      }
     }
    }, 15000);
  }
});

bot.on('death', () => {
  addChatLog('Bot tử vong! Chờ hồi sinh...');
  addErrorLog('Event Chết', 'Bot tử vong');

  respawnTimer = setTimeout(() => {
    if (bot && bot._client && !isManualStopped) {
      try { bot.respawn(); } catch (e) {}
    }
    setTimeout(() => {
      if (bot && bot.entity && bot._client && !isManualStopped) {
        executeActiveFeaturesOnSpawn();
        scheduleNextAction();
        scheduleRandomRotation();
      }
    }, 4500);
  }, 4000);
});

bot.on('message', (message) => {
  try {
    const text = message.toString().trim();
    if (!text) return;
    const lowerText = text.toLowerCase();

    if (lowerText.includes(BOT_USERNAME.toLowerCase())) {
      addBotMentionLog(text);
    }

    if (text.includes('') || lowerText.includes('hồi chiêu') || lowerText.includes('cooldown')) return;

    addChatLog(text);
  } catch (e) {}
});

bot.on('end', (reason) => {
  stopFeatureLoops();
  addErrorLog('Mất Kết Nối (End)', `Server ngắt socket: ${reason}`);
```

```
    handleReconnect();
  });

  bot.on('error', (err) => {
    stopFeatureLoops();
    addErrorLog('Mineflayer Error', err.message || err.toString());
  });

  bot.on('kicked', (reason) => {
    stopFeatureLoops();
    const reasonStr = typeof reason === 'string' ? reason : JSON.stringify(reason);
    addErrorLog('Bị Server Kick', reasonStr);

    if (reasonStr.includes('IP') || reasonStr.includes('đã có người') || reasonStr.includes('LƯU DỮ
LIỆU')) {
      currentReconnectDelay = 25000;
    }
    handleReconnect();
  });
}

function handleReconnect() {
  if (isManualStopped || isReconnecting) return;
  isReconnecting = true;
  cleanupBot();

  consecutiveFailures++;

  if (consecutiveFailures >= 12) {
    addErrorLog('CẢNH BÁO NẶNG', 'Mất kết nối liên tục 12 lần. Khởi động lại App...');
    setTimeout(() => { process.exit(1); }, 3000);
    return;
  }

  const jitter = Math.floor(Math.random() * 3000);
  const waitTime = currentReconnectDelay + jitter;

  console.log(`Chờ ${Math.round(waitTime / 1000)}s để tái kết nối...`);

  reconnectTimeout = setTimeout(() => {
    isReconnecting = false;
    currentReconnectDelay = Math.min(currentReconnectDelay * 1.25, 30000);
    createBot();
```

```
  }, waitTime);
}

createBot();

process.on('uncaughtException', (err) => {
  addErrorLog('Uncaught Exception', `${err.message} (${err.code || 'NO_CODE'})`);
  try { handleReconnect(); } catch(e) {}
});

process.on('unhandledRejection', (reason) => {
  addErrorLog('Unhandled Rejection', String(reason));
});
```